

WEEK8

BOOKFLOW LIBRARY SYSTEM

Transactional Microservice

Spring Boot + Spring Data JPA + PostgreSQL

Aim:

The Transactional Microservice (Spring Boot + JPA) Scenario The "BookFlow" library needs a strict system to manage Inventory (how many copies of a book are on the shelf) and Orders (who borrowed what and when). Because these involve financial-style accuracy, we use Spring Boot and PostgreSQL. You will build this service and connect it to your existing ecosystem. Task to be Done

1. Spring Boot Project Setup • Use Spring Initializr to create a Maven project with dependencies: Spring Web, Spring Data JPA, PostgreSQL Driver, and Validation. • Configure application.yml to connect to your PostgreSQL instance.
2. Entity Mapping (The Relational Model) • Inventory Entity: Create a class to track bookId (from the FastAPI service), totalStock, and availableStock. • Order Entity: Create a class to track userId (from the Node.js service), bookId, and status (BORROWED/RETURNED). • Use JPA Annotations (@Entity, @Table, @Id, @Column) to map these to database tables.
3. The Service Layer (CRUD & Logic) • Repository: Create interfaces extending JpaRepository for zero-code CRUD. • Service: Write logic to check if availableStock > 0 before allowing a user to "Borrow" a book. • Controller: Implement @PostMapping for /orders and @GetMapping for /inventory/{bookId}.
4. Validation & Error Handling • Use JSR-380 annotations (like @Min(0)) to ensure stock never goes below zero. • Implement a @ControllerAdvice class to return a clean JSON error message if a book is out of stock.
5. Gateway Integration • Update your FastAPI Gateway to route requests to this new service. • The Final Test: A user requests a book via the Gateway → Gateway calls Spring Boot → Spring Boot checks stock and records the order.

Skill Set Gained • Strict Layering: Understanding the industry-standard Controller-Service-Repository pattern. • Type Safety: Using Java's strong typing and Spring's Dependency Injection (IoC) for reliable code. • Automated Persistence: Leveraging Spring Data JPA to interact with SQL without writing raw queries. • Polyglot Harmony: Managing a system where Python (FastAPI), JavaScript (Node.js), and Java (Spring Boot) all work together.

Solution:

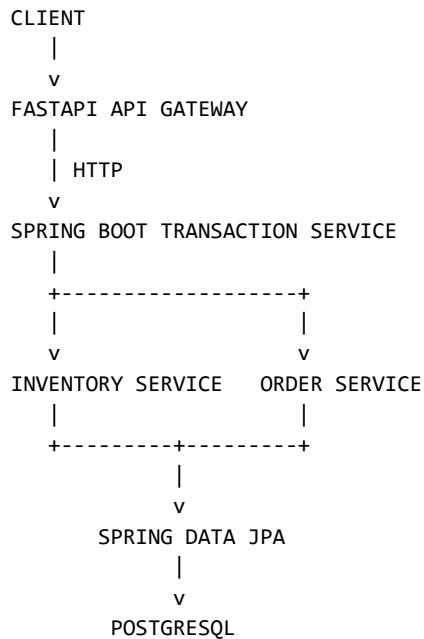
1. Project Objective

BookFlow manages book inventory and borrowing orders. The Spring Boot service checks available stock, creates an order, and reduces inventory within a database transaction. PostgreSQL stores the persistent data.

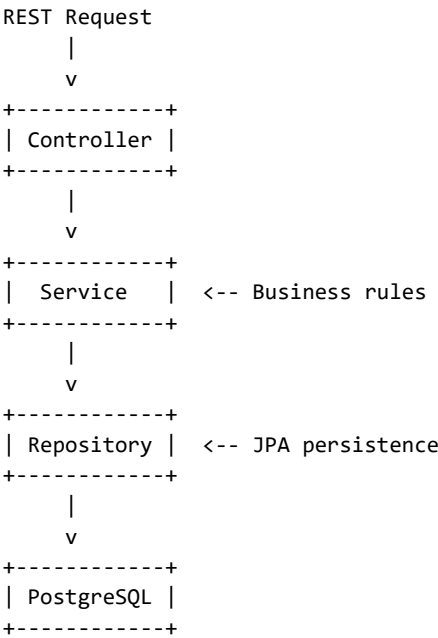
2. Technology Stack

Technology	Purpose
Java 17+	Application language
Spring Boot	Microservice framework
Spring Web	REST APIs
Spring Data JPA	Database persistence
PostgreSQL	Relational database
Maven	Build and dependency management
Postman	API testing
FastAPI	Gateway integration
VS Code / IntelliJ / Eclipse	Development IDE

3. Architecture Dashboard



4. Internal Layered Architecture



5. Project Structure

```
bookflow-transaction/
├── pom.xml
├── src/main/java/com/bookflow/transaction/
│   ├── BookflowTransactionApplication.java
│   ├── controller/
│   │   ├── OrderController.java
│   │   └── InventoryController.java
│   ├── entity/
│   │   ├── Inventory.java
│   │   └── Order.java
│   ├── repository/
│   │   ├── InventoryRepository.java
│   │   └── OrderRepository.java
│   ├── service/
│   │   ├── InventoryService.java
│   │   └── OrderService.java
│   ├── dto/
│   │   └── OrderRequest.java
│   └── exception/
│       ├── BookOutOfStockException.java
│       └── GlobalExceptionHandler.java
└── src/main/resources/
    └── application.yml
```

6. Database Model

INVENTORY

```
-----
book_id          PRIMARY KEY
total_stock      >= 0
available_stock  >= 0
```

ORDERS

```
-----
id               PRIMARY KEY
user_id
book_id
status           BORROWED / RETURNED
```

7. PostgreSQL Setup

```
CREATE DATABASE bookflow;

-- After connecting to bookflow:
SELECT current_database();
```

Expected:

```
bookflow
```

8. application.yml

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/bookflow
    username: postgres
```

password: postgres

jpa:

hibernate:

ddl-auto: update

show-sql: true

properties:

hibernate:

format_sql: true

server:

port: 8080

9. Main Program

```
package com.bookflow.transaction;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class BookflowTransactionApplication {
    public static void main(String[] args) {
        SpringApplication.run(
            BookflowTransactionApplication.class, args);
    }
}
```

10. Inventory Entity

```
@Entity
@Table(name = "inventory")
public class Inventory {

    @Id
    private Long bookId;

    @Min(0)
    private Integer totalStock;

    @Min(0)
    private Integer availableStock;

    // constructors, getters and setters
}
```

11. Order Entity

```
@Entity
@Table(name = "orders")
public class Order {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private Long userId;
    private Long bookId;
    private String status;

    // constructors, getters and setters
}
```

12. Repository Programs

```
public interface InventoryRepository
    extends JpaRepository<Inventory, Long> {
}
```

```
public interface OrderRepository
    extends JpaRepository<Order, Long> {
}
```

JpaRepository supplies save(), findById(), findAll(), delete(), existsById() and other CRUD operations.

13. Order Request DTO

```
public class OrderRequest {

    @NotNull
    private Long userId;

    @NotNull
    private Long bookId;

    // getters and setters
}
```

14. Transactional Service Logic

```
@Service
public class OrderService {

    @Transactional
    public Order borrowBook(OrderRequest request) {

        Inventory inventory =
            inventoryRepository.findById(request.getBookId())
                .orElseThrow(() ->
                    new RuntimeException("Book not found"));

        if (inventory.getAvailableStock() <= 0) {
            throw new BookOutOfStockException(
                "Book " + request.getBookId()
                + " is out of stock");
        }

        inventory.setAvailableStock(
            inventory.getAvailableStock() - 1);

        inventoryRepository.save(inventory);

        Order order = new Order(
            request.getUserId(),
            request.getBookId(),
            "BORROWED");

        return orderRepository.save(order);
    }
}
```

15. Controller Programs

```
@RestController
@RequestMapping("/inventory")
public class InventoryController {

    @GetMapping("/{bookId}")
    public Inventory getInventory(
        @PathVariable Long bookId) {
        return inventoryService.getInventory(bookId);
    }
}

@RestController
@RequestMapping("/orders")
public class OrderController {

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Order borrowBook(
        @Valid @RequestBody OrderRequest request) {
        return orderService.borrowBook(request);
    }
}
```

16. Global Error Handling

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(BookOutOfStockException.class)
    @ResponseStatus(HttpStatus.CONFLICT)
    public Map<String, Object> handleOutOfStock(
        BookOutOfStockException ex) {

        return Map.of(
            "status", 409,
            "error", "OUT_OF_STOCK",
            "message", ex.getMessage());
    }
}
```

17. Dashboard – Initial Database State

The following dashboard represents the expected PostgreSQL state after inserting sample inventory.

Book ID	Total Stock	Available	Borrowed	Stock Status
101	5	5	0	AVAILABLE
102	3	3	0	AVAILABLE
103	2	0	2	OUT OF STOCK

Visual stock indicator:

Book 101 5 available

Book 102 3 available

Book 103 0 available

18. Insert Sample Data

```
INSERT INTO inventory
(book_id, total_stock, available_stock)
VALUES
(101, 5, 5),
(102, 3, 3),
(103, 2, 0);

SELECT * FROM inventory;
```

19. Dashboard – After One Borrow

Metric	Before	After	Change
Book 101 total stock	5	5	0
Book 101 available	5	4	-1
Book 101 borrowed	0	1	+1
Orders created	0	1	+1

Book 101: 4 available / 5 total

Borrowed: 1 order

20. How to Run the Project – Step by Step

Step 1 – Install Java

Install JDK 17 or later. Verify with:

```
java -version
```

Step 2 – Verify Maven

Verify Maven with:

```
mvn -version
```

Step 3 – Start PostgreSQL

Ensure PostgreSQL service is running.

Step 4 – Create Database

Create database named bookflow.

Step 5 – Open Project

Open the generated Maven project in VS Code or IntelliJ.

Step 6 – Configure Database

Edit application.yml with your PostgreSQL username/password.

Step 7 – Build Project

Run Maven build.

```
mvn clean install
```

Step 8 – Start Spring Boot

Run the application.

```
mvn spring-boot:run
```

Step 9 – Check Server

Open the API using Postman.

```
http://localhost:8080
```

Step 10 – Insert Test Data

Insert sample inventory using pgAdmin/psql.

Step 11 – Test GET

Call GET /inventory/101.

Step 12 – Test POST

Call POST /orders with userId and bookId.

Step 13 – Verify Database

Query inventory and orders tables.

21. Expected Console Output

```
Tomcat initialized with port 8080
Started BookflowTransactionApplication
Hibernate: create table inventory ...
Hibernate: create table orders ...
Tomcat started on port 8080
```

22. Postman Testing – GET Inventory

Item	Value
Method	GET
URL	http://localhost:8080/inventory/101
Body	None

GET http://localhost:8080/inventory/101

Expected response:

```
{
  "bookId": 101,
  "totalStock": 5,
  "availableStock": 5
}
```

23. Postman Testing – Borrow Book

Item	Value
Method	POST
URL	http://localhost:8080/orders
Header	Content-Type: application/json

```
{
  "userId": 501,
  "bookId": 101
}
```

Expected response:

```
{
  "id": 1,
  "userId": 501,
  "bookId": 101,
  "status": "BORROWED"
}
```

24. Verify Inventory After Borrowing

GET http://localhost:8080/inventory/101

```
{
  "bookId": 101,
  "totalStock": 5,
  "availableStock": 4
}
```

25. Verify PostgreSQL

```
SELECT * FROM inventory;
SELECT * FROM orders;
```

book id		total stock	available stock
101		5	4
102		3	3
103		2	0
id	user id	book id	status
1	501	101	BORROWED

26. Out-of-Stock Test

If availableStock is zero, the service must not create an order.

```
POST http://localhost:8080/orders
```

```
{
  "userId": 502,
  "bookId": 103
}
```

Expected HTTP response: 409 Conflict

```
{
  "status": 409,
  "error": "OUT_OF_STOCK",
  "message": "Book 103 is out of stock"
}
```

27. Validation Test

Stock values must not be negative. @Min(0) rejects values below zero.

```
@Min(value = 0, message = "Available stock cannot be negative")
private Integer availableStock;
```

28. End-to-End Gateway Dashboard

```
CLIENT
|
| POST /gateway/orders
| { userId: 501, bookId: 101 }
v
FASTAPI GATEWAY
|
| HTTP POST /orders
v
SPRING BOOT
|
+--> Find Inventory
|
+--> availableStock > 0 ?
|
|      YES
|      |
|      +--> availableStock - 1
|      |
|      +--> Create BORROWED Order
|
v
POSTGRESQL
|
v
JSON RESPONSE
{ id: 1, userId: 501, bookId: 101, status: "BORROWED" }
```

29. FastAPI Gateway Example

```
from fastapi import FastAPI
import httpx

app = FastAPI()

SPRING_SERVICE = "http://localhost:8080"

@app.post("/gateway/orders")
async def create_order(order: dict):

    async with httpx.AsyncClient() as client:
        response = await client.post(
            f"{SPRING_SERVICE}/orders",
            json=order
        )

    return response.json()
```

30. Complete Testing Checklist

Test	Expected Result	Status
Spring Boot starts	Port 8080 active	PASS
PostgreSQL connection	Application connects	PASS
GET inventory	Returns stock JSON	PASS
POST order	Creates BORROWED order	PASS
Stock decrement	Available stock decreases by 1	PASS
Zero stock	HTTP 409	PASS
Negative stock validation	Rejected	PASS
Database persistence	Rows remain in PostgreSQL	PASS
Gateway forwarding	FastAPI reaches Spring Boot	PASS

31. Common Problems and Solutions

Problem	Likely Cause	Solution
Connection refused	PostgreSQL not running	Start PostgreSQL service
Password authentication failed	Wrong DB password	Update application.yml
Port 8080 in use	Another service uses 8080	Stop it or change server.port
Book not found	No inventory row	Insert the book into inventory
Out of stock	availableStock = 0	Add stock or choose another book
Maven command not found	Maven not installed/PATH issue	Install Maven or use Maven wrapper

32. Learning Outcomes

- Controller–Service–Repository architecture
- Spring dependency injection and IoC
- REST API development
- Spring Data JPA and automatic CRUD
- PostgreSQL database integration
- Validation with @Min and @NotNull
- Global exception handling
- Transactional business logic using @Transactional
- Postman API testing
- FastAPI-to-Spring Boot microservice communication
- Polyglot architecture using Python, JavaScript and Java

33. Final Command Sheet

```
# Verify Java
java -version
```

```
# Verify Maven
mvn -version
```

```
# Build
mvn clean install
```

```
# Run
mvn spring-boot:run
```

```
# API
GET http://localhost:8080/inventory/101
```

```
POST http://localhost:8080/orders
```

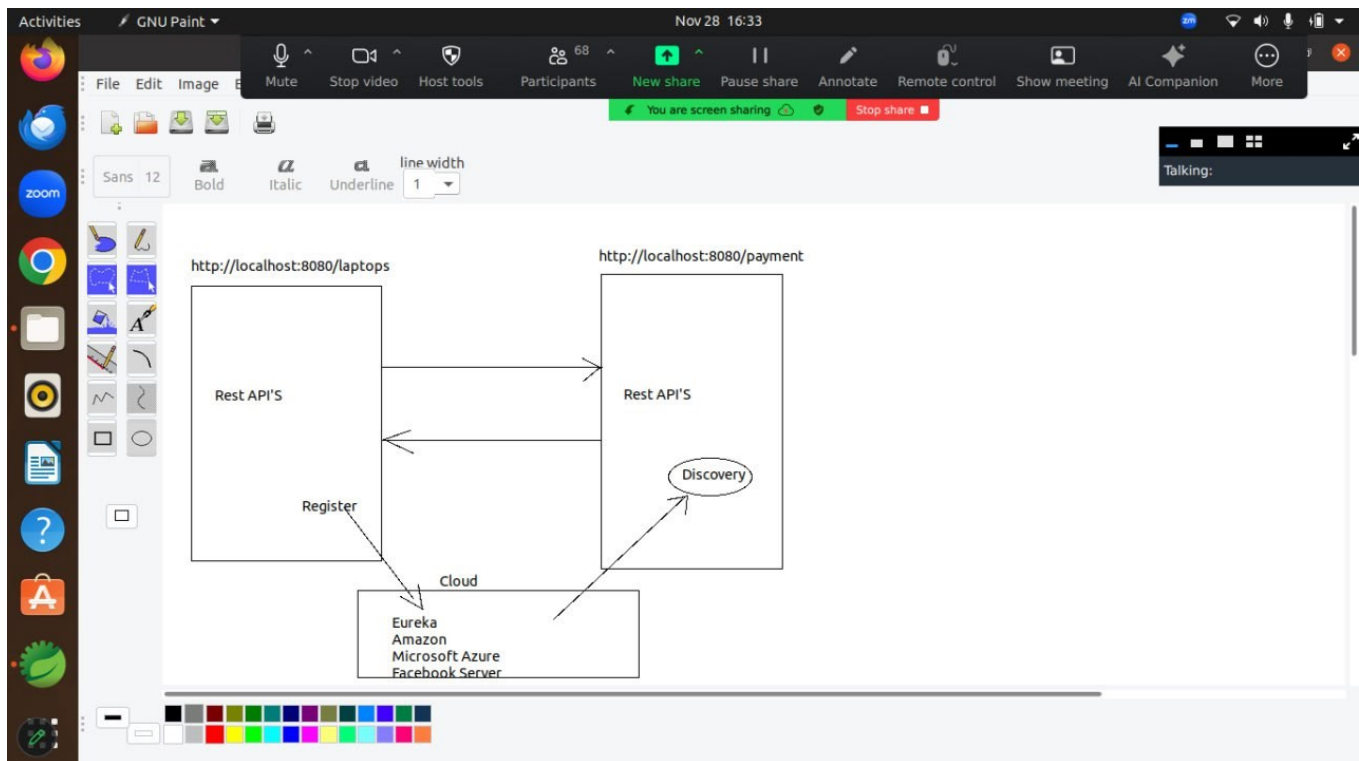
```
# PostgreSQL  
CREATE DATABASE bookflow;  
  
# Verify data  
SELECT * FROM inventory;  
SELECT * FROM orders;
```

One more example for practice:

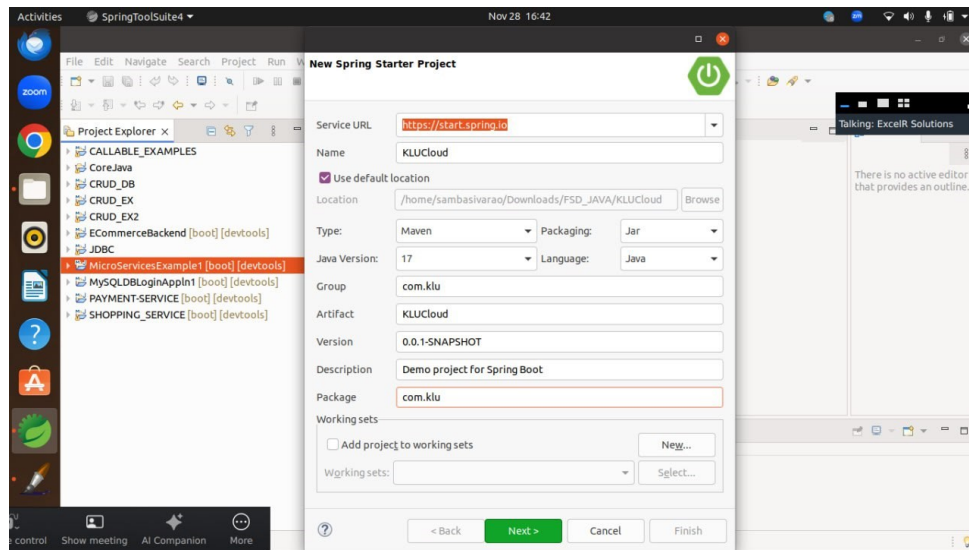
Microservices using spring boot

Creation of Eureka Server

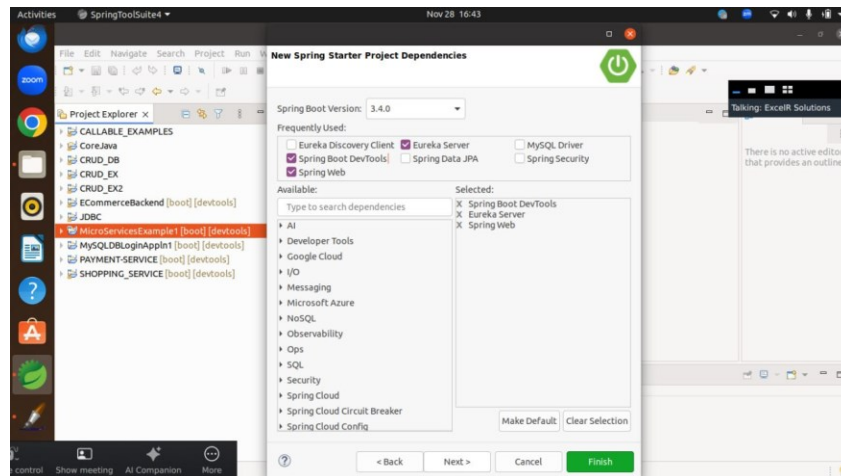
Step-1



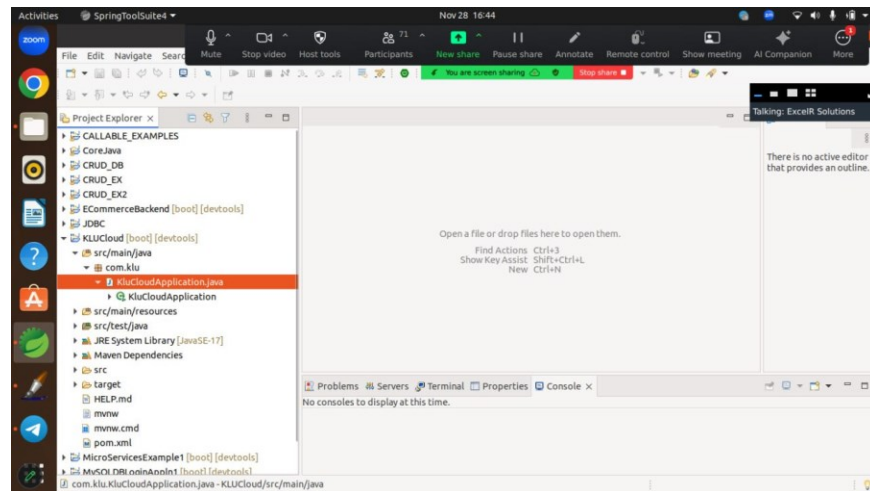
Step-2



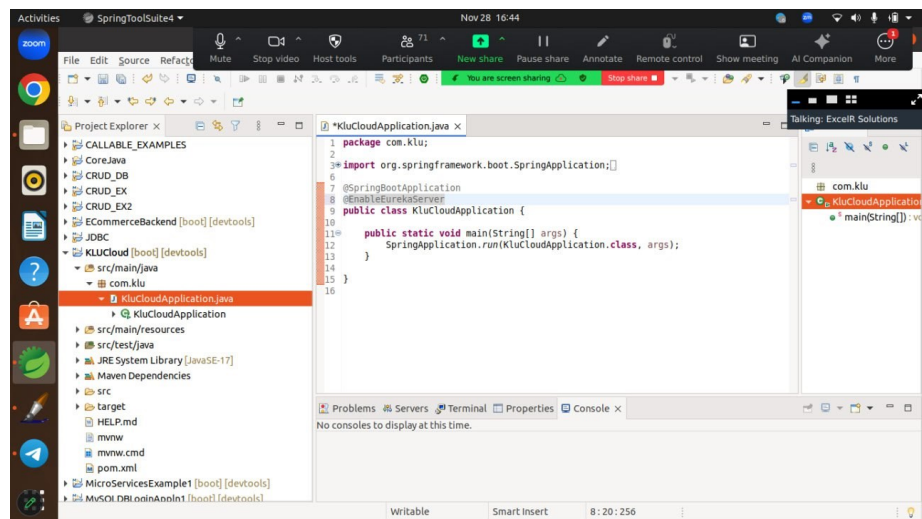
Step-3



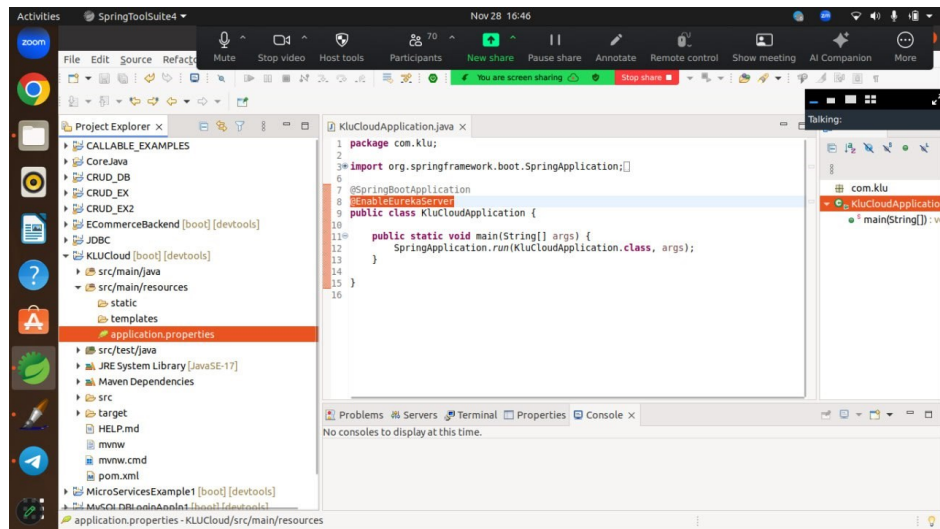
Step-4



Step-5



Step-6



Step-7

Change application.properties file to application.yml

Application.yml

eureka:

client:

register-with-eureka: false

fetch-registry: false

server:

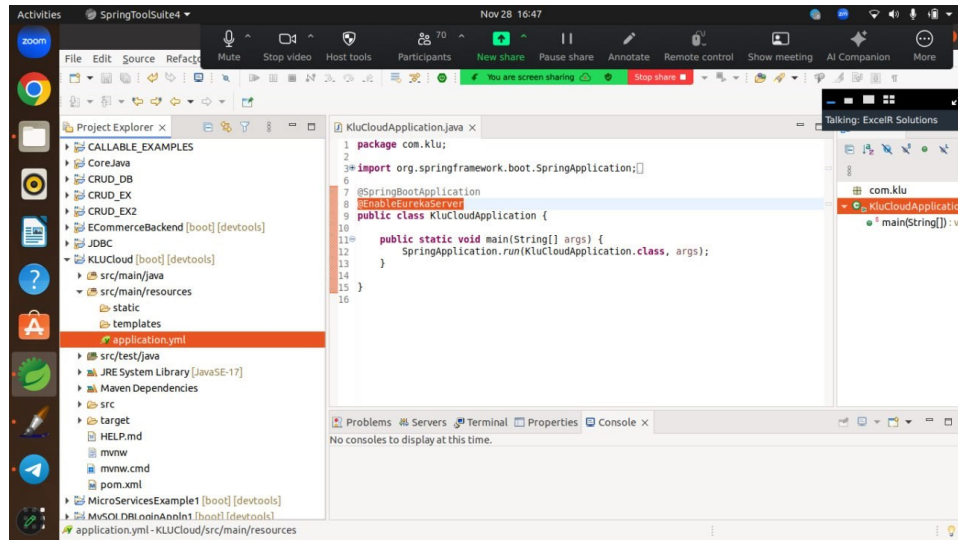
port: 8761

spring:

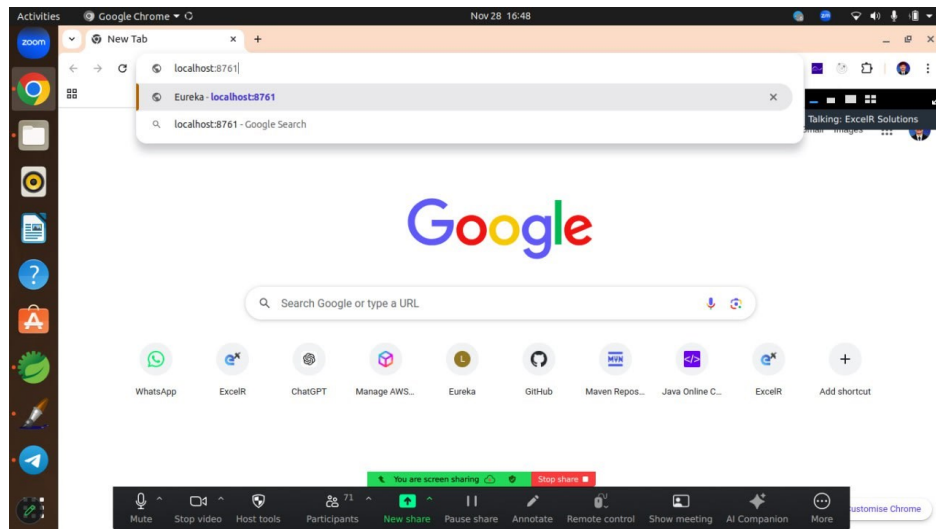
application:

name: EUREKA-CLOUD-SERVER

Step-8



Step-9



Step-10

Activities

Google Chrome

Nov 28 16:49

zoom

Eureka

localhost:8761

spring Eureka

HOME LAST 1000 SJI Talking: ExcelR Solutions

System Status

Environment	test	Current time	2024-11-28T16:48:48 +0530
Data center	default	Uptime	00:00
		Lease expiration enabled	false
		Renews threshold	1
		Renews (last min)	0

DS Replicas

localhost

Insta

You are screen sharing

Stop share

Mute

Stop video

Host tools

Participants

New share

Pause share

Annotate

Remote control

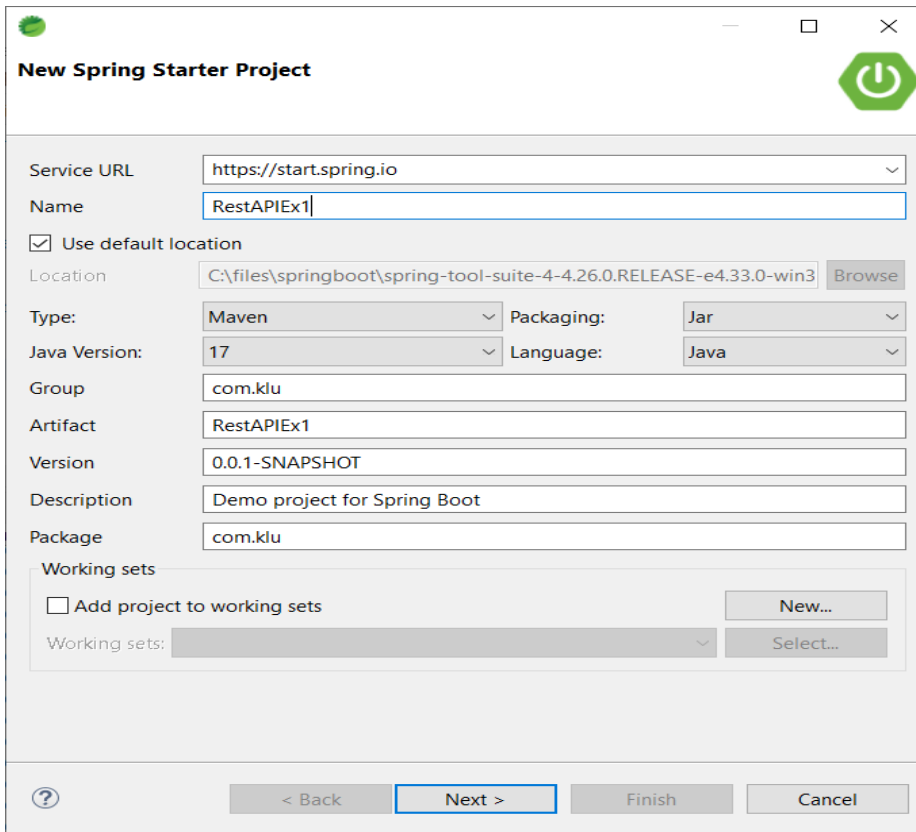
Show meeting

AI Companion

More

Creation RESTApi project and run in Eureka Server

Step-1



The screenshot shows the 'New Spring Starter Project' dialog box. The 'Service URL' is set to 'https://start.spring.io'. The 'Name' is 'RestAPIEx1'. The 'Use default location' checkbox is checked, and the 'Location' is 'C:\files\springboot\spring-tool-suite-4-4.26.0.RELEASE-e4.33.0-win3'. The 'Type' is 'Maven', 'Packaging' is 'Jar', 'Java Version' is '17', and 'Language' is 'Java'. The 'Group' is 'com.klu', 'Artifact' is 'RestAPIEx1', 'Version' is '0.0.1-SNAPSHOT', 'Description' is 'Demo project for Spring Boot', and 'Package' is 'com.klu'. The 'Working sets' section has an unchecked checkbox 'Add project to working sets' and a 'New...' button. The 'Working sets:' dropdown is empty, and there is a 'Select...' button. At the bottom, there are buttons for '< Back', 'Next >', 'Finish', and 'Cancel'.

New Spring Starter Project

Service URL:

Name:

☒ Use default location

Location:

Type: Packaging:

Java Version: Language:

Group:

Artifact:

Version:

Description:

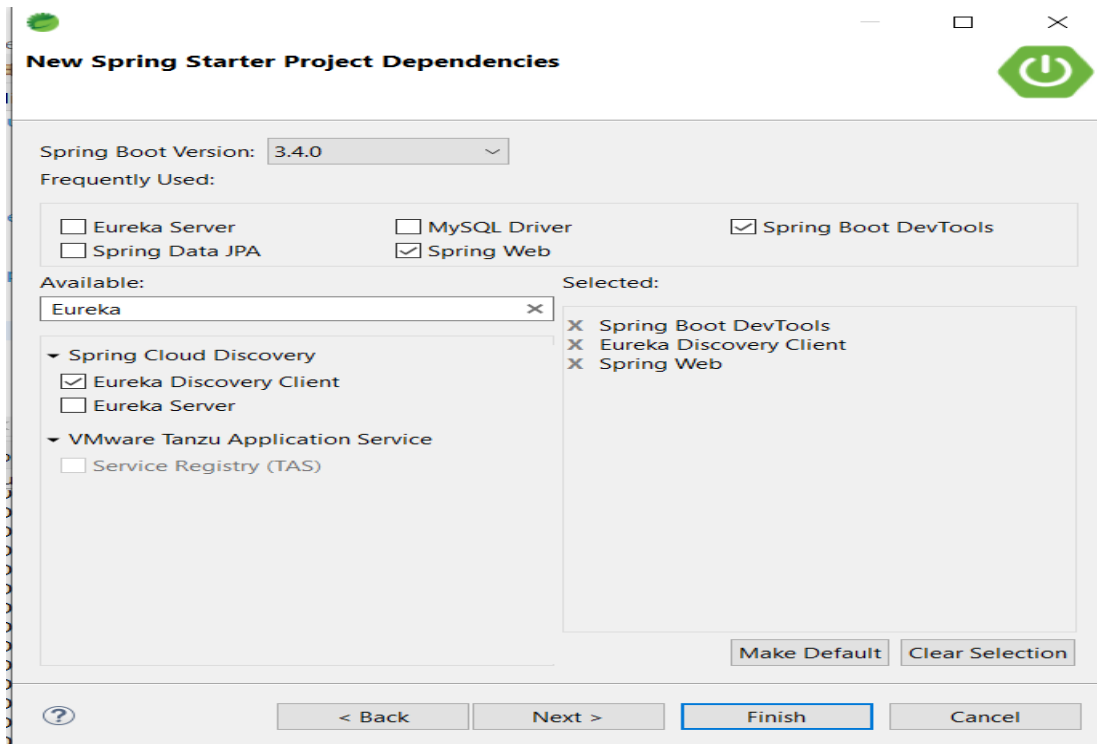
Package:

Working sets

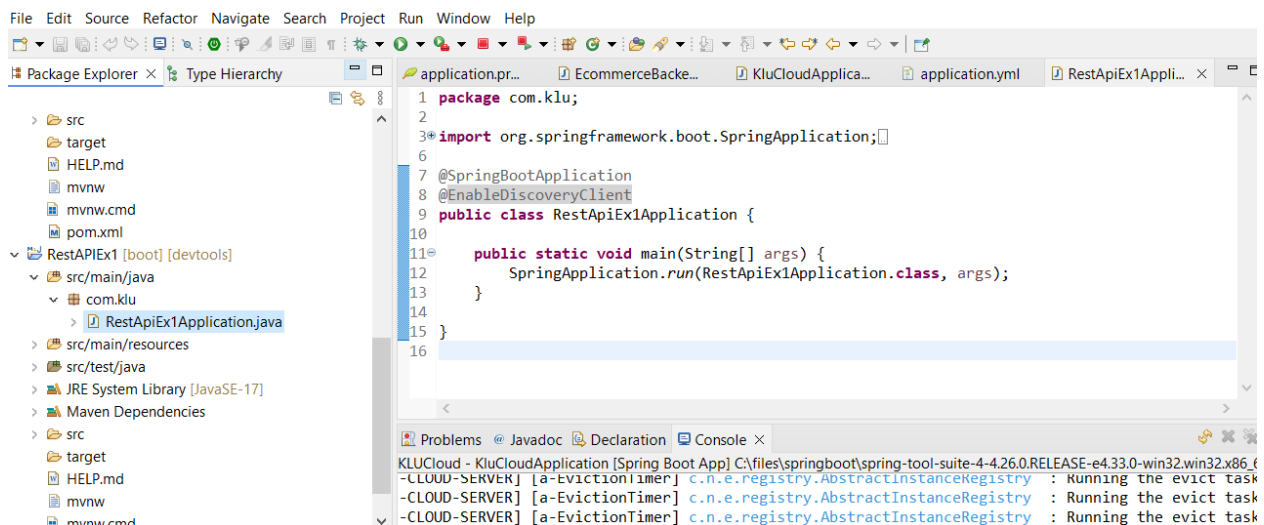
☐ Add project to working sets

Working sets:

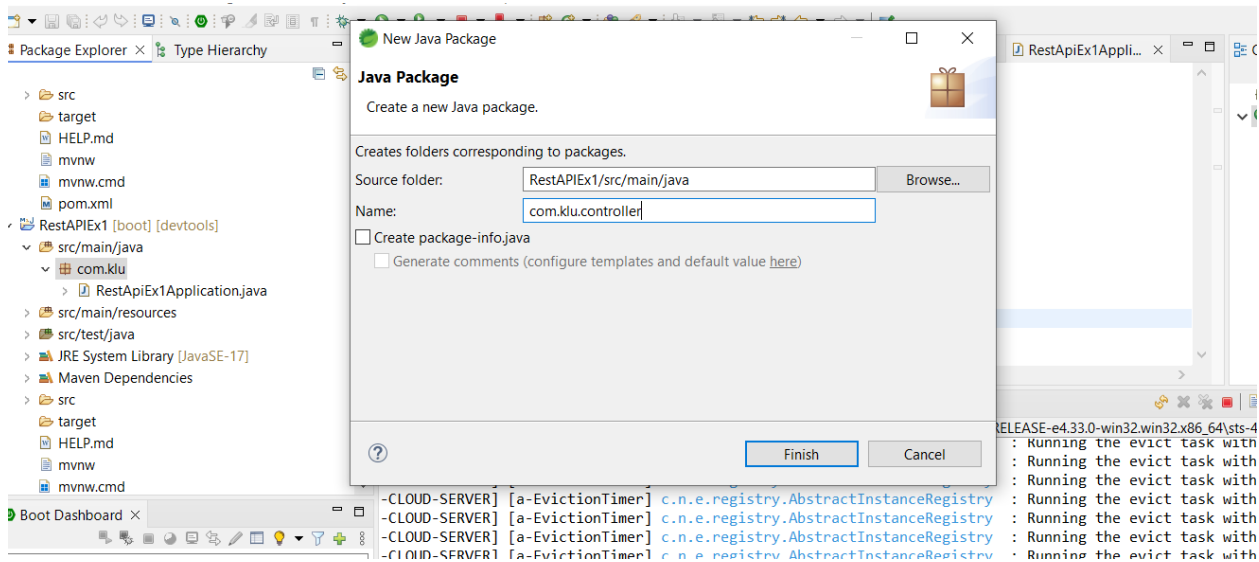
Step-2



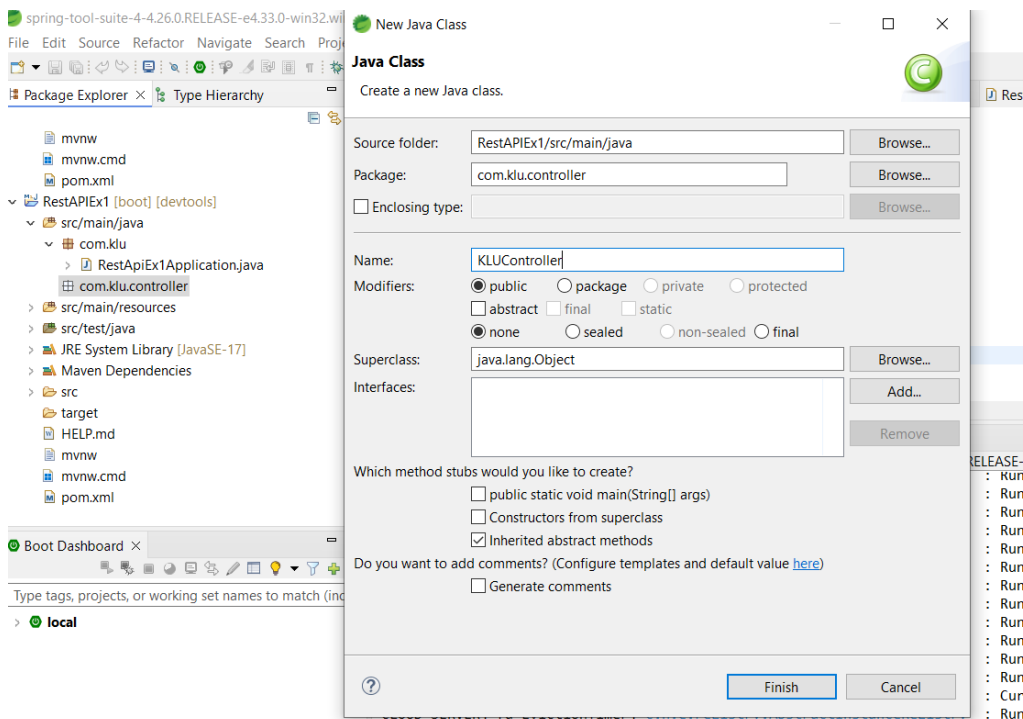
Step-3



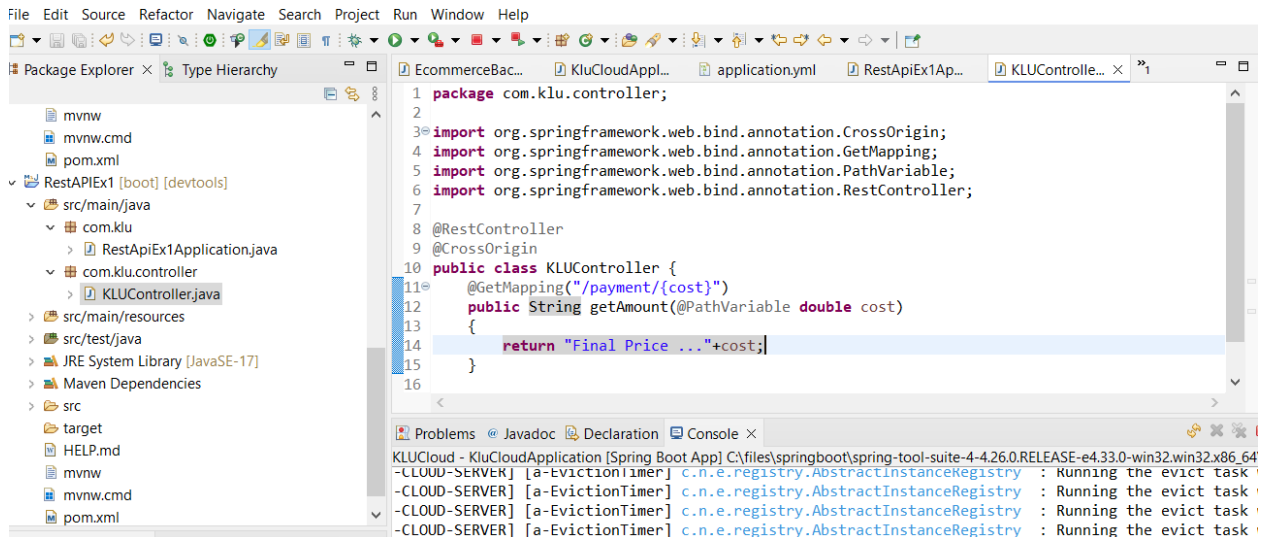
Step-4



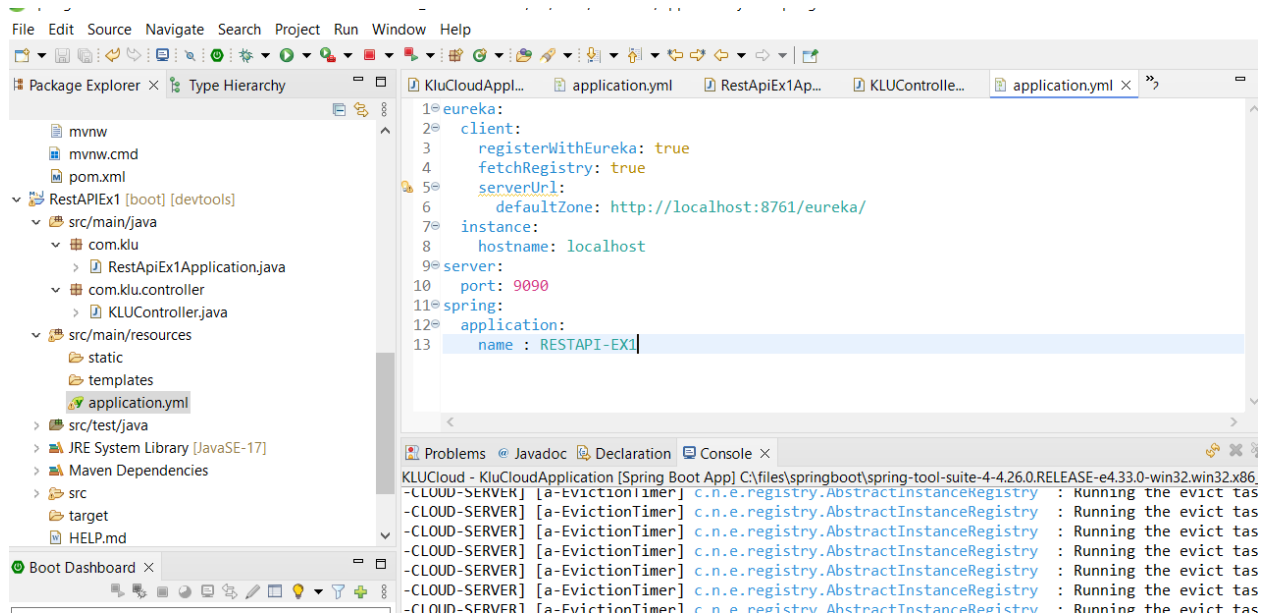
Step-5



Step-6



Step-7



Application.yml
eureka:

client:
 registerWithEureka: true
 fetchRegistry: true
 serverUrl:
 defaultZone: http://localhost:8761/eureka/
instance:
 hostname: localhost
server:
 port: 8001
spring:
 application:
 name : RESTAPI-EX2

Step-8

← → ↺ localhost:8761 🔍 ☆ ⬇ ⚠

🗄 | 📖 Software Testing Help ⚙️ KL ERP 🌐 Scientific Internatio... 🌐 portal.klef.in/klef 🗣 Bypass | HIX.AI ⚙️ KLH - Learning Man... 🔔 Notifications | Linke... » | 📁 All Br

[localhost](#)

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
RESTAPI-EX1	n/a (1)	(1)	UP (1) - DESKTOP-MCL27LN-RESTAPI-EX1-9090

General Info

Name	Value
total-avail-memory	88mb
num-of-cpus	4
current-memory-usage	59mb (67%)
server-uptime	00:39
registered-replicas	http://localhost:8761/eureka/
unavailable-replicas	http://localhost:8761/eureka/,
available-replicas	

Microservices project: Deploy RestAPIEx2 Project in RestAPIEx1

RestAPIEx2:

 — □ × 

New Spring Starter Project

Service URL:

Name:

☒ Use default location

Location:

Type: Packaging:

Java Version: Language:

Group:

Artifact:

Version:

Description:

Package:

Working sets

☐ Add project to working sets

Working sets:

?

Step-2:

 — □ × 

New Spring Starter Project Dependencies

Spring Boot Version:

Frequently Used:

☒ Eureka Discovery Client	☐ Eureka Server	☐ MySQL Driver
☒ Spring Boot DevTools	☐ Spring Data JPA	☒ Spring Web

Available:

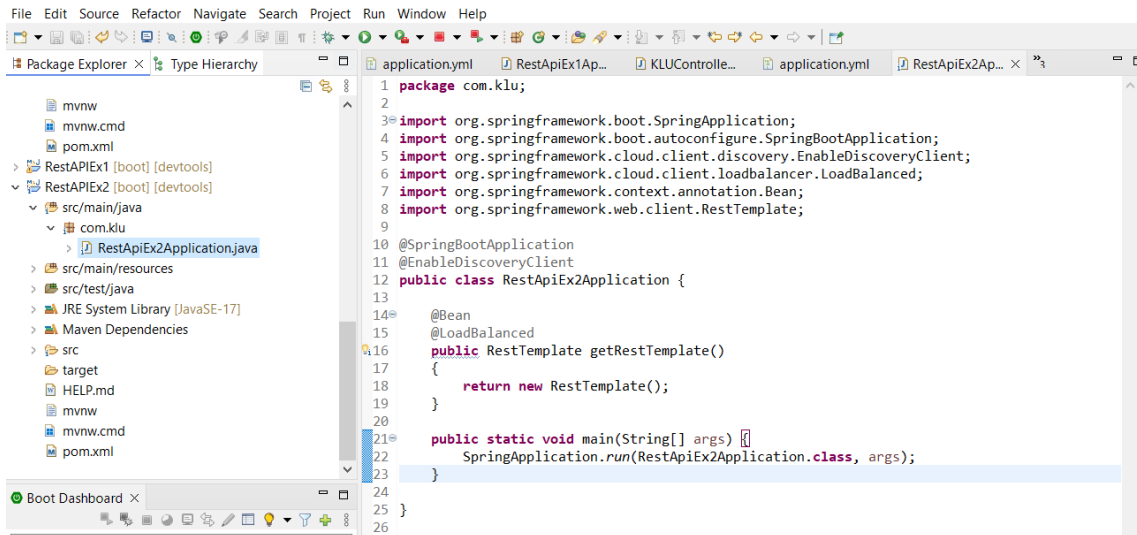
- ▶ AI
- ▶ Developer Tools
- ▶ Google Cloud
- ▶ I/O
- ▶ Messaging
- ▶ Microsoft Azure
- ▶ NoSQL
- ▶ Observability
- ▶ Ops
- ▶ SQL
- ▶ Security

Selected:

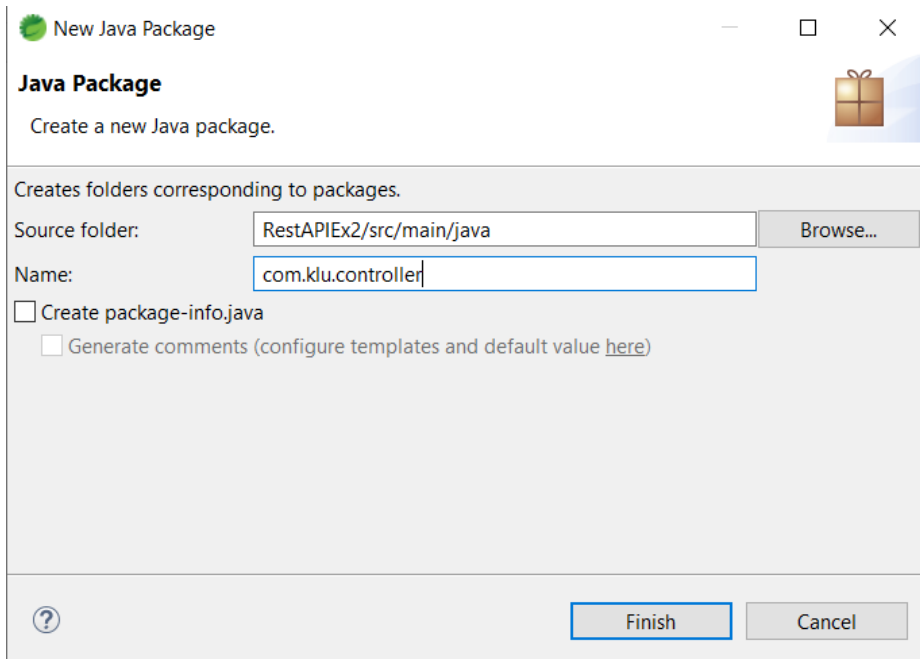
- X Spring Boot DevTools
- X Eureka Discovery Client
- X Spring Web

?

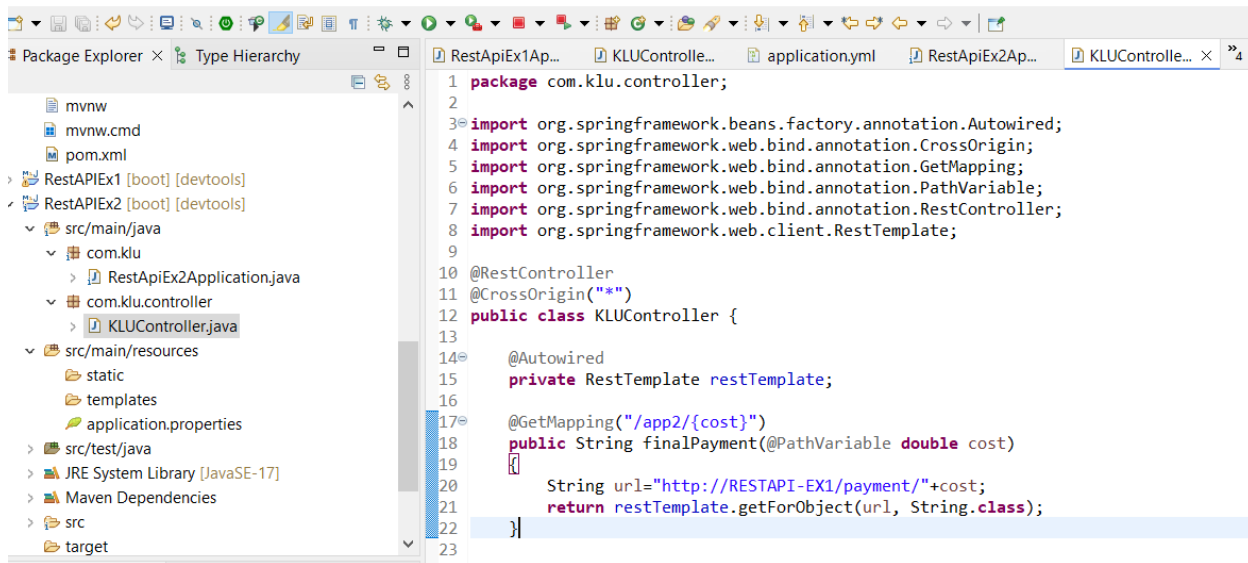
Step-3:



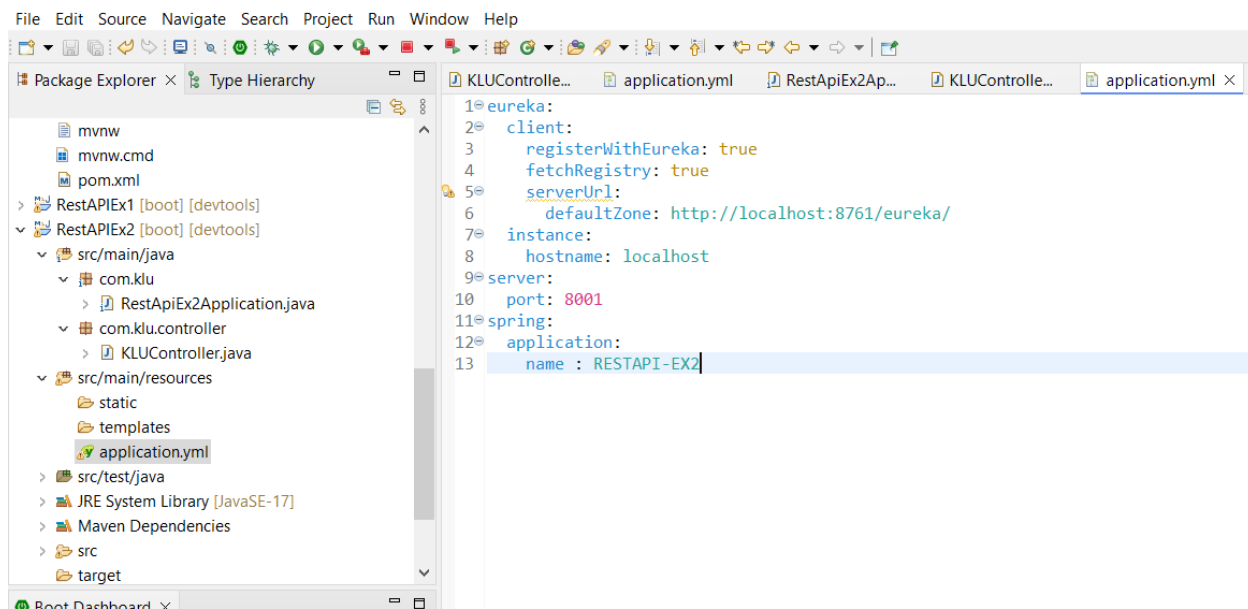
Step-4



Step-5



Step-6



Step-7

localhost:8761

Software Testing HelpKL ERPScientific Internatio...portal.klef.in/klefBypass | HIXAIKLH - Learning Man...Notifications | Linke...All Bookm

DS Replicas

localhost

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
RESTAPI-EX1	n/a (1)	(1)	UP (1) - DESKTOP-MCL27LN:RESTAPI-EX1:9090
RESTAPI-EX2	n/a (1)	(1)	UP (1) - DESKTOP-MCL27LN:RESTAPI-EX2:8001

Step-8

My WorkspaceNew ImportOverviewGET userregisterNo environment

CollectionsNew CollectionPOST userregister

OverviewGET userregister

ParamsAuthorizationHeaders (8)BodyScriptsSettings

noneform-datax-www-form-urlencodedrawbinaryGraphQLJSON

```
1 {
2   "username": "kluhzayudu",
3   "password": "subbu1234",
4   "email": "subbu1@gmail.com",
5   "role": "fullstack"
6 }
```

BodyCookiesHeaders (8)Test Results200 OK4.24 s273 B

PrettyRawPreviewVisualizeText

1 Final Price ...500.00

